Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе

«Анализ накопления ошибок при суммировании рядов Тейлора»

Выполнил:
студент группы 3825Б1ПМии1
Соснов С. С.

Проверила:

Бусько П.В.

Нижний Новгород
2026

Содержание

Введение 3
Постановка задачи 4
Руководство пользователя 5
Описание программной реализации 7
Результаты экспериментов 9
Заключение 17
Литература 18
Приложение 19

Введение

Известно, что наиболее простыми функциями в смысле вычисления являются многочлены. Возникает вопрос о возможности замены функции f в окрестности точки x многочленом некоторой степени. Представить функцию в виде многочлена позволяет ряд Тейлора (Рис. 1), который даёт возможность вычислить значение функции приближённо, суммируя члены ряда.

Однако при реализации этого метода на компьютере члены ряда становятся числами с плавающей точкой, что вносит погрешность округления. При этом порядок суммирования ряда может существенно повлиять на итоговую точность. Данная работа направлена на исследование влияния порядка суммирования членов ряда Тейлора на точность вычислений при работе с числами с плавающей точкой, а также на изучение тонкостей работы с этими числами для максимизации точности.

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots + \frac{x^n}{n!} + o(x^n).$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots + (-1)^{m-1}\frac{x^{2m-1}}{(2m-1)!} + o(x^{2m}).$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots + (-1)^m\frac{x^{2m}}{(2m)!} + o(x^{2m+1}).$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots + (-1)^{n-1}\frac{x^n}{n} + o(x^n).$$

Рис. 1. Разложения для важнейших элементарных функций

Постановка задачи

Цель:

Реализовать функцию, которая будет вычислять значения функций (sin, cos, exp, ln(1+x)) с помощью рядов Тейлора в конкретной точке, суммируя ряд с начала и конца, и исследовать влияние порядка суммирования на точность вычислений.

Необходимо:

1. Реализовать вычисление членов ряда с использованием рекуррентных формул для каждой функции (Рис. 2).

2. Написать два алгоритма суммирования: от первого члена ряда до последнего и от последнего к первому.

3. Вычислить погрешность каждого метода, сравнивая эталонное значение с полученной суммой.

4. Провести эксперименты для различных значений аргумента и количества членов ряда.

5. Построить графики зависимости ошибки от порядка суммирования и проанализировать, в каких случаях значение имеет максимальную точность.

$$\sin(x) \quad x_n = (-1)\frac{x^2}{(2n)(2n+1)}x_{n-1}$$

$$cos(x) \quad x_n = (-1)\frac{x^2}{(2n)(2n-1)}x_{n-1}$$

$$e^x \quad x_n = \frac{x}{n}x_{n-1}$$

$$ln(x+1) \quad x_n = (-1)\frac{x(n-1)}{n}x_{n-1}$$

Рис. 2. Рекуррентные формулы для каждой из функций

Руководство пользователя

Данная программа позволяет пользователю построить ряд Тейлора с произвольным количеством членов, вычислить значение функции в желаемой точке, а также вывести значение погрешностей при вычислении. В текущей версии программа выполняется один раз и завершает работу.

1. При запуске программы появится возможность выбора количества членов ряда. Необходимо *ввести желаемое количество членов* в ряде Тейлора.

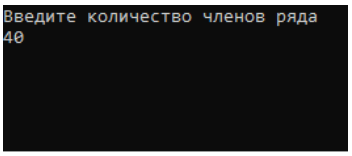

Рис. 3. Пример ввода количества членов

2. После выбора количества членов, пользователю будет предложено выбрать точку разложения. Необходимо *ввести желаемое значение аргумента функции*.

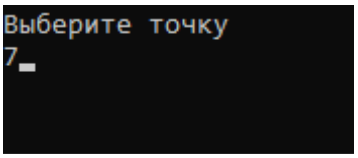

Рис. 4. Пример ввода точки разложения

3. Далее появится меню выбора функций.

   Для *вычисления синуса в точке* – нажмите цифру 1.

   Для *вычисления косинуса в точке* – нажмите цифру 2.

   Для *вычисления экспоненты в точке* – нажмите цифру 3.

   Для *вычисления логарифма в точке* – нажмите цифру 4.

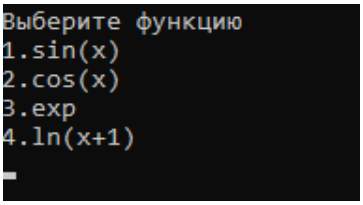

Рис. 5. Меню выбора функций

4. В конце будет выведен результат всех вычислений.

   В строке *Ryad* будут последовательно выведены все члены ряда Тейлора для выбранной функции.

   В строке *StartZnach* будет выведено конечно значение функции, полученное путём суммирования ряда от первого челна до последнего.

   В строке *StartSumm* будут последовательно выведены частичные суммы, где каждый i-ый элемент это сумма первых i членов ряда. Суммирование идёт с первого члена до последнего.

В строке *StartErrors* будут выведены ошибки вычисления для каждого шага суммирования, вычисляемый как разность эталонного значения с каждым элементом строки *StartSumm.*

В строке *EndZnach* будет выведено конечно значение функции, полученное путём суммирования ряда от последнего члена до первого.

В строке *EndSumm* будут последовательно выведены частичные суммы, где каждый *i*-ый элемент это сумма последних *i* членов ряда. Суммирование идёт с последнего члена ряда до первого.

В строке *EndErrors* будут выведены ошибки вычисления для каждого шага суммирования, вычисляемый как разность эталонного значения с каждым элементом строки *EndSumm.*

```
Ryad
1.000000  7.000000  24.500000  57.166664  100.041664  140.058334  163.401382  163.401382  142.976212  111.203720

StartZnach
910.749329

StartSumm
1.000000  8.000000  32.500000  89.666664  189.708328  329.766663  493.168030  656.569397  799.545593  910.749329

StartErrors
1095.633179  1088.633179  1064.133179  1006.966492  906.924866  766.866516  603.465149  440.063782  297.087585  185.883850

EndZnach
910.749390

EndSumm
111.203720  254.179932  417.581299  580.982666  721.041016  821.082703  878.249390  902.749390  909.749390  910.749390

EndErrors
985.429443  842.453247  679.051880  515.650513  375.592163  275.550476  218.383789  193.883789  186.883789  185.883789
```

Рис. 6. Пример вывода результатов всех вычислений

Описание программной реализации

Проект состоит из 10 файлов, 5 заголовочных файлов, 4 файла реализации и главный файл программы. В главном файле программы находится исходный код приложения на языке Си. Четыре из пяти заголовочных файлов имеют одноимённые файлы реализации. Файл Ryad.h объявляет структуру Xn с шестью полями: количество слагаемых ряда, массив слагаемых ряда, массив частичных сумм, считаемых с первого элемента до последнего, массив ошибок для первого вида суммирования, массив частичных сумм, считаемых с последнего элемента до первого, массив ошибок для второго вида суммирования. Файлы MainFunc.h и MainFunc.cpp содержат прототипы и реализацию функций, использующих рекуррентные формулы для каждой стандартной функции:

1. `float rSin(float prev, float x, int n)` – вычисляет n-ый член ряда для синуса по рекуррентной формуле, зная (n-1)-ый. Принимает на вход предыдущий член ряда, точку разложения, номер текущего члена, возвращает значение n-го (текущего) члена ряда. Начальные значения: при n = 0 возвращает x (первый член ряда).
2. `float rCos(float prev, float x, int n)` – вычисляет n-ый член ряда для косинуса по рекуррентной формуле, зная (n-1)-ый. Принимает на вход предыдущий член ряда, точку разложения, номер текущего члена, возвращает значение n-го (текущего) члена ряда. Начальные значения: при n = 0 возвращает 1 (первый член ряда).
3. `float rExp(float prev, float x, int n)` – вычисляет n-ый член ряда для экспоненты по рекуррентной формуле, зная (n-1)-ый. Принимает на вход предыдущий член ряда, точку разложения, номер текущего члена, возвращает значение n-го (текущего) члена ряда. Начальные значения: при n = 0 возвращает 1 (первый член ряда).
4. `float rLn(float prev, float x, int n)` – вычисляет n-ый член ряда для ln(x+1) по рекуррентной формуле, зная (n-1)-ый. Принимает на вход предыдущий член ряда, точку разложения, номер текущего члена, возвращает значение n-го (текущего) члена ряда. Начальные значения: при n = 0 возвращает 0 (первый член ряда), при n = 1 возвращает x (второй член ряда).

Файлы Init.h и Init.cpp содержат прототипы и реализацию функций, инициализирующих структуру ряда и освобождающих память:

1. `void Init(struct Xn* r, int n)` – инициализирует структуру ряда. Принимает на вход указатель на структуру, которую будет инициализировать, и количество слагаемых в ряде, чтобы выделить память на все массивы в структуре и заполнить их нулями. Функция ничего не возвращает.
2. `void FreeR(struct Xn* r)` – освобождает память, выделенную под массивы структуры, присваивает NULL всем указателям структуры, обнуляет поле с числом слагаемых. Принимает на вход указатель на структуру. Функция ничего не возвращает

Файлы Filling.h и Filling.cpp содержат прототипы и реализацию функций, заполняющих поля структуры:

1. `void Fill(struct Xn* r, float(*function)(float prev, float x, int n), float x)` – заполняет массив слагаемых структуры с помощью цикла, используя рекуррентную формулу функции. Принимает на вход указатель на структуру, в которой будет заполнятся массив, указатель на функцию из файла MainFunc, точку разложения. Функция ничего не возвращает
2. `void Sum(struct Xn* r)` – заполняет два массива частичных сумм структуры с помощью цикла, где *i*-ый элемент, это сумма (*i*-1)-го элемента и *i* –го члена ряда. Принимает на вход указатель на структуру ряда. Функция ничего не возвращает.
3. `void Errors(struct Xn* r, float(*etalon)(float x), float x)` – заполняет два массива ошибок для каждого из алгоритмов суммирования, каждый элемент массива вычисляется как абсолютное значение разности эталонного значения с элементом того же номера массива частичных сумм. Принимает на вход указатель на структуру ряда, указатель на функцию, которая считает эталонное значение, указатель на функцию из файла MainFunc. Функция ничего не возвращает.
4. `void choices(int choice, struct Xn* r, float x)` – служит для экономии места в программе, она реализует логику выбора функции в зависимости от ввода пользователя. Принимает на вход номер выбранной функции, указатель на структуру ряда, точку разложения. Эта функция создаёт два нулевых указателя на функцию: один на эталонную, другой на рекуррентную, и, в зависимости от входного значения, присваивает указателям значения. Далее вызываются функции Fill, Sum, Errors для ряда и выбранных функций. Функция нечего не возвращает.

Файлы PrintRyad.h и PrintRyad.cpp содержат прототип и реализацию функции `void PrintR(struct Xn* r)`, которая выводит на экран (в консоль) результаты вычислений. Принимает на вход указатель на структуру ряда. Функция ничего не возвращает.

Результаты экспериментов

В ходе работы были проведены эксперименты, для каждой функции выбирались два значения точки разложения: малое и большое. Эксперименты должны были показать, как значение x влияет на накопление ошибок при вычислении ряда Тейлора, как порядок суммирования влияет на точность, какие ограничения накладывает тип float при работе с большими значениями x. Для самых показательных примеров были построены графики.

Sin(x)

Для начала вводились значения, при которых синус известен, например, x = 1.5708 или x = 0.5236, чтобы проверить корректность работы программы. После проверки корректности был проведён эксперимент для малых значений x, был выбран x = 0.5, а n = 10 (т.к. дальше слагаемые зануляются). Вывод работы программы приведён на Рис. 7.1.

```
Ryad
0.500000  -0.020833  0.000260  -0.000002  0.000000  -0.000000  0.000000  -0.000000  0.000000  -0.000000

StartZnach
0.479426

StartSumm
0.500000  0.479167  0.479427  0.479426  0.479426  0.479426  0.479426  0.479426  0.479426  0.479426

StartErrors
0.020574  0.000259  0.000002  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000

EndZnach
0.479426

EndSumm
-0.000000  0.000000  -0.000000  0.000000  -0.000000  0.000000  -0.000002  0.000259  -0.020574  0.479426

EndErrors
0.479426  0.479426  0.479426  0.479426  0.479426  0.479426  0.479427  0.479167  0.500000  0.000000
```

Рис. 7.1

По результатам вывода видно, что при x < 1.0 слагаемые очень быстро зануляются и становятся настолько малыми, что не помещаются в 6 цифр после точки. Если использовать

```
Ryad
5.00e-01  -2.08e-02  2.60e-04  -1.55e-06  5.38e-09  -1.22e-11  1.96e-14  -2.33e-17  2.14e-20  -1.57e-23  9.3
3e-27  -4.61e-30  1.92e-33  -6.84e-37  2.11e-40  -5.61e-44  0.00e+00  -0.00e+00  0.00e+00  -0.00e+00

StartZnach
0.4794255197

StartSumm
5.00e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01
  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01

StartErrors
2.06e-02  2.59e-04  1.52e-06  2.98e-08  2.98e-08  2.98e-08  2.98e-08  2.98e-08  2.98e-08  2.98e-08  2.98e-08
  2.98e-08  2.98e-08  2.98e-08  2.98e-08  2.98e-08  2.98e-08  2.98e-08  2.98e-08  2.98e-08

EndZnach
0.4794255495

EndSumm
-0.00e+00  0.00e+00  0.00e+00  0.00e+00  -5.61e-44  2.11e-40  -6.84e-37  1.92e-33  -4.61e-30  9.33e-27  -1.5
7e-23  2.14e-20  -2.33e-17  1.96e-14  -1.22e-11  5.37e-09  -1.54e-06  2.59e-04  -2.06e-02  4.79e-01

EndErrors
4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01
  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01  4.79e-01  5.00e-01  0.00e+00
```

Рис. 7.2

вывод чисел в научной нотации и увеличить количество слагаемых, то получим Рис. 7.2.

Здесь наглядно видно, что 17 член ряда равен 0.00e+00, то есть округляется до нуля. На самом деле 17 член ряда представляется как (0.5^33) / (33!), что примерно равно 1.34e-47, такое число не входит в диапазон значений типа данных float, что и вызывает округление до нуля. Также заметно, что значения, полученные разными способами суммирования, очень похожи, но не равны. Более точным нужно считать второе значение, полученное суммированием с конца ряда, т.к. разность последней частичной суммы с эталонным значением настолько мала, что также округляется до нуля, в отличие от *StartSumm*, где уже после 4 слагаемого, последующие значения настолько малы, что не меняют сумму.

Далее был проведён эксперимент для большого значения x = 12. Результаты вывода видны на Рис. 7.3.

```
Ryad
1.20000e+01  -2.88000e+02  2.07360e+03  -7.10949e+03  1.42190e+04  -1.86139e+04  1.71821e+04  -1.17820e+04  6.23753e+03  -2.62633e+03  9.00455e+02  -2.56256e+02  6.15015e+01  -1.26157e+01  2.23726e+00
  -3.46415e-01  4.72384e-02  -5.71625e-03  6.17973e-04  -6.00459e-05  5.27232e-06  -4.20385e-07  3.05734e-08  -2.03634e-09  1.24674e-10  -7.04042e-12  3.67859e-13  -1.78356e-14  8.04614e-16  -3.38587e
-17  1.33214e-18  -4.91113e-20  1.70001e-21  -5.53598e-23  1.69902e-24  -4.92272e-26  1.34869e-27  -3.49930e-29  8.61073e-31  -2.01224e-32  4.47165e-34  -9.46103e-36  1.90811e-37  -3.67238e-39  6.7520
2e-41  -1.18690e-42  1.96182e-44  -0.00000e+00  0.00000e+00  -0.00000e+00

StartZnach
-0.5366694927

StartSumm
1.20000e+01  -2.76000e+02  1.79760e+03  -5.31189e+03  8.90709e+03  -9.70684e+03  7.47524e+03  -4.30676e+03  1.93077e+03  -6.95555e+02  2.04901e+02  -5.13554e+01  1.01461e+01  -2.46960e+00  -2.32339e-0
1  -5.78754e-01  -5.31516e-01  -5.37232e-01  -5.36614e-01  -5.36674e-01  -5.36669e-01  -5.36670e-01  -5.36669e-01  -5.36669e-01  -5.36669e-01  -5.36669e-01  -5.36669e-01  -5.36669e-01  -
5.36669e-01  -5.36669e-01  -5.36669e-01  -5.36669e-01  -5.36669e-01  -5.36669e-01  -5.36669e-01  -5.36669e-01  -5.36669e-01  -5.36669e-01  -5.36669e-01  -5.36669e-01  -5.36669e-01  -5.36669e-01  -5.36
669e-01  -5.36669e-01  -5.36669e-01  -5.36669e-01  -5.36669e-01  -5.36669e-01  -5.36669e-01

StartErrors
1.25366e+01  2.75463e+02  1.79814e+03  5.31135e+03  8.90762e+03  9.70630e+03  7.47578e+03  4.30622e+03  1.93131e+03  6.95018e+02  2.05437e+02  5.08188e+01  1.06827e+01  1.93303e+00  3.04234e-01  4.218
16e-02  5.05686e-03  6.59406e-04  4.14252e-05  1.01447e-04  9.62019e-05  9.66191e-05  9.65595e-05  9.65595e-05  9.65595e-05  9.65595e-05  9.65595e-05  9.65595e-05  9.65595e-05  9.65595e-05  9.65595e-0
5  9.65595e-05  9.65595e-05  9.65595e-05  9.65595e-05  9.65595e-05  9.65595e-05  9.65595e-05  9.65595e-05  9.65595e-05  9.65595e-05  9.65595e-05  9.65595e-05  9.65595e-05  9.65595e-05  9.
65595e-05  9.65595e-05  9.65595e-05  9.65595e-05

EndZnach
-0.5371093750

EndSumm
-0.00000e+00  0.00000e+00  0.00000e+00  1.96182e-44  -1.16728e-42  6.63529e-41  -3.60603e-39  1.87205e-37  -9.27383e-36  4.37891e-34  -1.96845e-32  8.41388e-31  -3.41516e-29  1.31454e-27  -4.79126e-26
  1.65111e-24  -5.37087e-23  1.64630e-21  -4.74650e-20  1.28468e-18  -3.25740e-17  7.72040e-16  -1.70636e-14  3.50796e-13  -6.68962e-12  1.17984e-10  -1.91836e-09  2.86551e-08  -3.91730e-07  4.88059e-
06  -5.51653e-05  5.62807e-04  -5.15344e-03  4.20850e-02  -3.04330e-01  1.93293e+00  -1.06827e+01  5.08187e+01  -2.05437e+02  6.95018e+02  -1.93131e+03  4.30622e+03  -7.47578e+03  9.70630e+03  -8.9076
2e+03  5.31135e+03  -1.79814e+03  2.75463e+02  -1.25371e+01  -5.37109e-01

EndErrors
5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.365
73e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36573e-01  5.36578e-01  5.36518e-0
1  5.37136e-01  5.31420e-01  5.78658e-01  2.32243e-01  2.46951e+00  1.01462e+01  5.13553e+01  2.04901e+02  6.95555e+02  1.93077e+03  4.30676e+03  7.47524e+03  9.70684e+03  8.90709e+03  5.31189e+03  1.
79760e+03  2.75999e+02  1.20005e+01  5.36442e-04
```

Рис. 7.3

По результатам вывода видно, что пиковое значение принимает 6 член ряда, после которого слагаемые начинают убывать, зануляясь после 48 члена ряда. Интересно, что в данном случае значение, полученное суммированием с начала, оказалось более точным, в отличие от второго. Сам массив *StartSumm* «стабилизировался» ближе к 19 элементу, после чего практически не менялся. В то же время в массиве *EndSumm* складывались числа, очень близкие к границе диапазона типа float, а соседние члены имели разные знаки, из-за чего сумма могла содержать большое количество ошибок, вызванных округлениями; когда сумма дошла до больших значений, они не могли повлиять на младшие разряды числа, из-за чего ошибка сохранилась. В *StartSumm* же более-менее точная сумма сформировалась в самом начале, из-за чего последующие малые значения, даже если и терялись, не давали такой большой потери точности.

Для этого эксперимента был построен график, отображающий, как ведут себя массивы ошибок, чтобы лучше понять, что происходит. На Рис. 7.4. видно, как *StartErrors* сначала растёт, достигая максимального значения при n = 6, что сходится с тем, что ряд сначала растёт, а потом начинает стремительно убывать. Интересно, что график для *EndErrors* выглядит довольно симметрично относительно первого, но ошибка уменьшалась не так стремительно, из-за чего конечное значение вышло не таким точным. Получается, что для суммирования ряда при малых x, предпочтительнее складывать члены с конца, т.к. ряд довольно быстро сходится и количество слагаемых не так велико, что не даёт ошибке накапливаться. В то же время, при больших значениях x, сначала слагаемые только возрастают, пока факториал в знаменателе не начнёт выравниваться с числителем. Также ряд сходится медленнее, из-за чего, суммируя его с конца, сумма членов, близких к границе диапазона, будет накапливать ошибку, а дойдя до больших значений, ошибка станет существенной, а последующие члены ряда не

смогут на это повлиять, так как будут слишком большими, из-за чего первый метод суммирования предпочтительней.

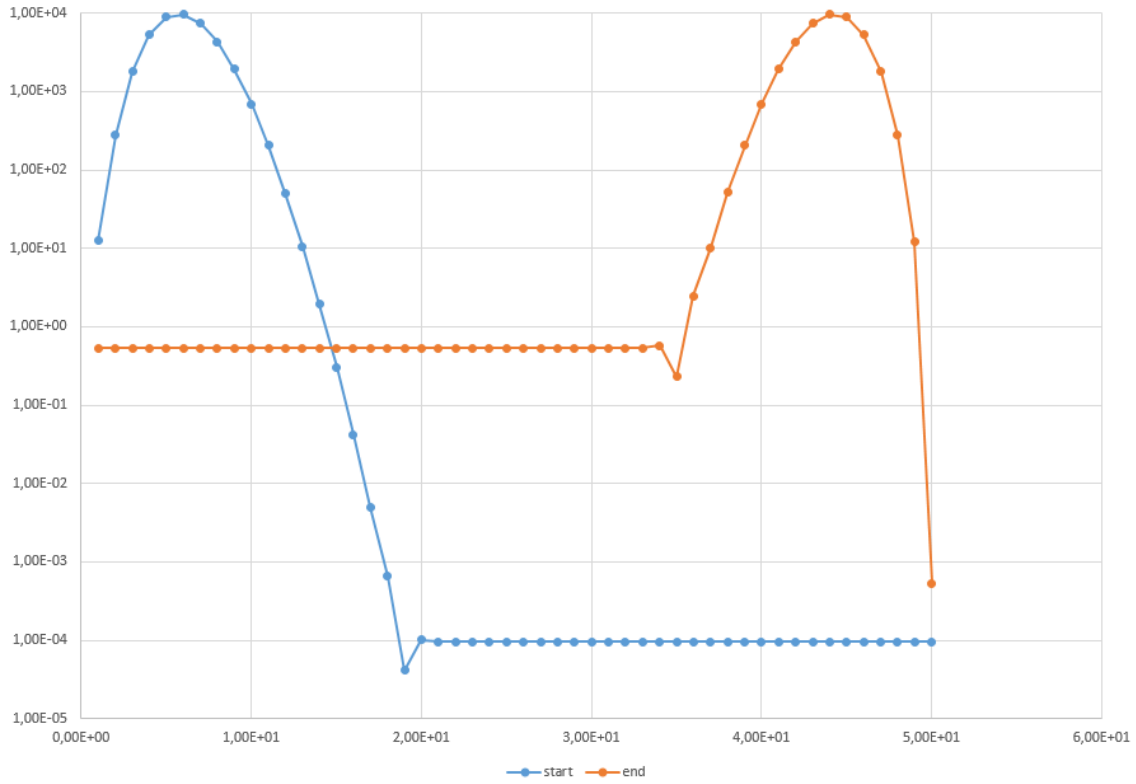

Рис. 7.4. График изменения ошибок при большом x для синуса

Cos(x)

Для косинуса план был тот же, как и для синуса, программа работала верно, давая приближённые значения, в которых косинус известен. Далее было выбраны две точки разложения: x = 0.3 и x = 10. Сначала вычислялось малое значение, вывод приведён на Рис. 8.1.

```
Ryad
1.00000e+00  -4.50000e-02  3.37500e-04  -1.01250e-06  1.62723e-09  -1.62723e-12  1.10948e-15  -5.48642e-19  2.05741e-22
-6.05120e-26  1.43318e-29  -2.79191e-33  4.55203e-37  -6.30276e-41  7.00649e-45  -0.00000e+00

StartZnach
0.9553364515

StartSumm
1.00000e+00  9.55000e-01  9.55337e-01  9.55336e-01  9.55336e-01  9.55336e-01  9.55336e-01  9.55336e-01  9.55336e-01  9.55
336e-01  9.55336e-01  9.55336e-01  9.55336e-01  9.55336e-01  9.55336e-01  9.55336e-01

StartErrors
4.46635e-02  3.36528e-04  9.53674e-07  5.96046e-08  5.96046e-08  5.96046e-08  5.96046e-08  5.96046e-08  5.96046e-08  5.96
046e-08  5.96046e-08  5.96046e-08  5.96046e-08  5.96046e-08  5.96046e-08  5.96046e-08

EndZnach
0.9553365111

EndSumm
-0.00000e+00  7.00649e-45  -6.30206e-41  4.55140e-37  -2.79145e-33  1.43290e-29  -6.04977e-26  2.05680e-22  -5.48437e-19
 1.10893e-15  -1.62612e-12  1.62561e-09  -1.01087e-06  3.36489e-04  -4.46635e-02  9.55337e-01

EndErrors
9.55337e-01  9.55337e-01  9.55337e-01  9.55337e-01  9.55337e-01  9.55337e-01  9.55337e-01  9.55337e-01  9.55337e-01  9.55
337e-01  9.55337e-01  9.55337e-01  9.55338e-01  9.55000e-01  1.00000e+00  0.00000e+00
```

Рис. 8.1.

Видно, что ситуация схожа с синусом, ряд также быстро сходится при достаточно малых значениях x, а значение, полученное суммированием ряда с конца, является более точным. В

массиве *StartSumm* частичная сумма перестаёт изменяться уже после 4 элемента, т.к. основная сумма уже будет сформирована, а последующие члены ряда, которые прибавляются, становятся настолько малыми, что не помещаются в мантиссу накопленной суммы, из-за чего их вклад теряется. В массиве *EndSumm* значения элементов постоянно меняются, вплоть до последнего, но в массиве *EndErrors* первые элементы одинаковые, т.к. значения частичных сумм очень малы, из-за чего вычитание суммы из элемента *EndErrors* никак не сказывается на значении, только накопив достаточный размер на 13 элементе, частичная сумма начала попадать в те самые 6-7 значащих цифр для числа ошибки. Если строить график (Рис. 8.2.) изменения значения ошибки при вычислении, то можно увидеть, как после четвёртого элемента значения *StartErrors* перестали уменьшаться, но всё равно были меньше, чем ошибки из *EndErrors*, только последнее значение стало достаточно точным.

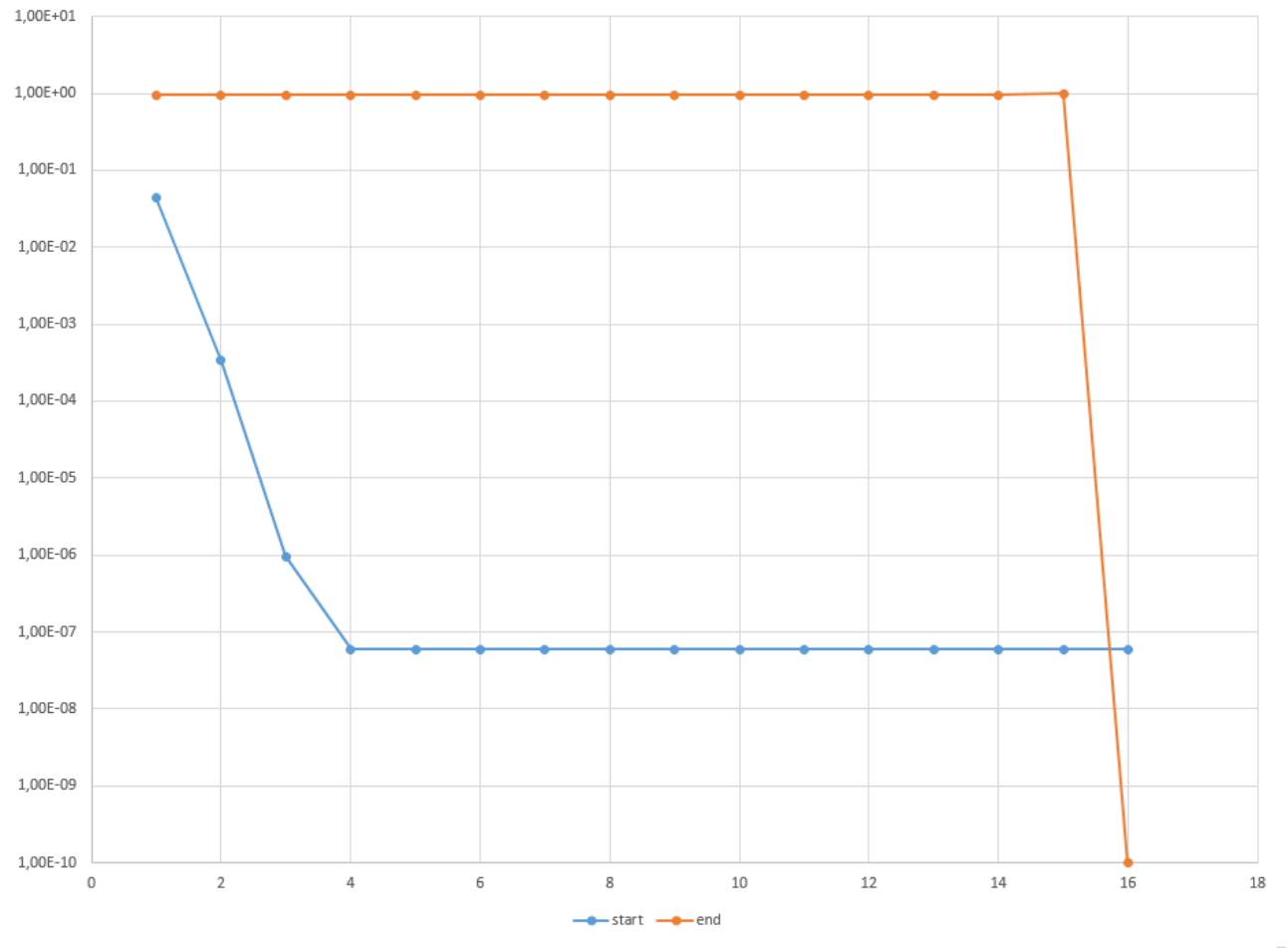

Рис. 8.2. График изменения ошибок при малом x для косинуса

При большом значении x, вывод приведён на Рис. 8.3. Здесь до сих пор алгоритм суммирования с конца имеет большую точность, но разрыв между значениями не такой большой, а ряд ведёт себя так же, как и синус, сначала абсолютное значение слагаемых достигает больших величин, а потом постепенно сходится. График очень похож на Рис. 7.4. Интересно, что при x = 11, как у косинуса, так и у синуса, разрыв между итоговыми значениями становится ещё меньше, а при x = 12, наоборот, алгоритм суммирования со старта становится приоритетнее, т.к. даёт более точный результат. Стоит отметить, что при действительно больших x (например, x =70) алгоритм для синуса и косинуса даёт абсолютно неправильные значения (sin (70) = -1929119011446477291520.0), которые выходят за область значений функций. Это происходит из-за того, что сумма пиковых значений ряда настолько большая, что остальные слагаемые не могут её скорректировать.

```
Ryad
1,0000e+00  -5,0000e+01  4,1667e+02  -1,3889e+03  2,4802e+03  -2,7557e+03  2,0877e+03  -1,1471e+03  4,7795e+02  -1,5619e+02  4,1103e+01  -8,8968e+00  1,6117e+00  -2,4796e-01
  3,2799e-02  -3,7700e-03  3,8004e-04  -3,3872e-05  2,6882e-06  -1,9120e-07  1,2256e-08  -7,1174e-10  3,7618e-11  -1,8173e-12  8,0555e-14  -3,2880e-15  1,2398e-16  -4,3319e-
18  1,4065e-19  -4,2543e-21  1,2018e-22  -3,1776e-24  7,8810e-26  -1,8371e-27  4,0322e-29  -8,3482e-31  1,6331e-32  -3,0231e-34  5,3036e-36  -8,8306e-38  1,3972e-39  -2,1036
e-41  3,0128e-43  -4,2039e-45  0,0000e+00

StartZnach
-0,8392691016

StartSumm
1,0000e+00  -4,9000e+01  3,6767e+02  -1,0212e+03  1,4589e+03  -1,2968e+03  7,9088e+02  -3,5619e+02  1,2175e+02  -3,4439e+01  6,6644e+00  -2,2324e+00  -6,2069e-01  -8,6865e-0
1  -8,3585e-01  -8,3962e-01  -8,3924e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01  -
8,3927e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01  -8,39
27e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01  -8,3927e-01

StartErrors
1,8391e+00  4,8161e+01  3,6851e+02  1,0204e+03  1,4598e+03  1,2960e+03  7,9172e+02  3,5536e+02  1,2259e+02  3,3600e+01  7,5034e+00  1,3934e+00  2,1838e-01  2,9575e-02  3,223
7e-03  5,4628e-04  1,6624e-04  2,0009e-04  1,9741e-04  1,9759e-04  1,9759e-04  1,9759e-04  1,9759e-04  1,9759e-04  1,9759e-04  1,9759e-04  1,9759e-04  1,9759e-04  1,9759e-04
  1,9759e-04  1,9759e-04  1,9759e-04  1,9759e-04  1,9759e-04  1,9759e-04  1,9759e-04  1,9759e-04  1,9759e-04  1,9759e-04  1,9759e-04  1,9759e-04  1,9759e-04  1,9759e-04  1,9
759e-04  1,9759e-04

EndZnach
-0,8392028809

EndSumm
0,0000e+00  -4,2039e-45  2,9708e-43  -2,0739e-41  1,3765e-39  -8,6929e-38  5,2167e-36  -2,9709e-34  1,6034e-32  -8,1879e-31  3,9503e-29  -1,7976e-27  7,7013e-26  -3,1006e-24
  1,1708e-22  -4,1372e-21  1,3651e-19  -4,1954e-18  1,1978e-16  -3,1682e-15  7,7387e-14  -1,7399e-12  3,5879e-11  -6,7586e-10  1,1580e-08  -1,7962e-07  2,5086e-06  -3,1363e-
05  3,4868e-04  -3,4213e-03  2,9378e-02  -2,1858e-01  1,3932e+00  -7,5036e+00  3,3600e+01  -1,2259e+02  3,5536e+02  -7,9172e+02  1,2960e+03  -1,4598e+03  1,0204e+03  -3,6851
e+02  4,8161e+01  -1,8392e+00  -8,3920e-01

EndErrors
8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,390
7e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3907e-01  8,3904e-01  8,3942e-01
  8,3565e-01  8,6845e-01  6,2049e-01  2,2322e+00  6,6646e+00  3,4439e+01  1,2175e+02  3,5619e+02  7,9088e+02  1,2968e+03  1,4589e+03  1,0212e+03  3,6767e+02  4,9000e+01  1,0
001e+00  1,3137e-04
```

Рис. 8.3.

Можно сделать вывод, что работа алгоритма для функций синуса и косинуса довольно похожа, ряд ведёт себя довольно схожим образом, что логично, так как рекуррентные формулы у них почти идентичны.

Exp

Экспонента интересна тем, что члены ряда функции всегда положительны, из-за чего вычисление значения будет проще. Предполагается, что алгоритм суммирования с конца ряда всегда будет давать более точный результат, так как здесь не будет сложностей с вычислением очень малых значений, которые почти равны, но отличаются знаком. Было проведено два эксперимента: при малых и больших значениях x, также была выполнена проверка, верно ли вычисляется функция. Для малого значения x было выбрано число 1, т.к. ряд быстро убывает, а полученное значение можно легко сверить. Результат работы программы приведён на Рис.

```
Ryad
1,0000e+00  1,0000e+00  5,0000e-01  1,6667e-01  4,1667e-02  8,3333e-03  1,3889e-03  1,9841e-04  2,4802e-05  2,7557e-06  2,7557e-07  2,5052e-08  2,0877e-09  1,60
59e-10  1,1471e-11  7,6472e-13  4,7795e-14  2,8115e-15  1,5619e-16  8,2206e-18  4,1103e-19  1,9573e-20  8,8968e-22  3,8682e-23  1,6117e-24  6,4470e-26  2,4796e-
27  9,1837e-29  3,2799e-30  1,1310e-31  3,7700e-33  1,2161e-34  3,8004e-36  1,1516e-37  3,3872e-39  9,6776e-41  2,6877e-42  7,2868e-44  1,4013e-45  0,0000e+00

StartZnach
2,7182819843

StartSumm
1,0000e+00  2,0000e+00  2,5000e+00  2,6667e+00  2,7083e+00  2,7167e+00  2,7181e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,71
83e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+
00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00

StartErrors
1,7183e+00  7,1828e-01  2,1828e-01  5,1615e-02  9,9483e-03  1,6148e-03  2,2602e-04  2,7657e-05  2,8610e-06  0,0000e+00  2,3842e-07  2,3842e-07  2,3842e-07  2,38
42e-07  2,3842e-07  2,3842e-07  2,3842e-07  2,3842e-07  2,3842e-07  2,3842e-07  2,3842e-07  2,3842e-07  2,3842e-07  2,3842e-07  2,3842e-07  2,3842e-07  2,3842e-
07  2,3842e-07  2,3842e-07  2,3842e-07  2,3842e-07  2,3842e-07  2,3842e-07  2,3842e-07  2,3842e-07  2,3842e-07  2,3842e-07  2,3842e-07  2,3842e-07

EndZnach
2,7182817459

EndSumm
0,0000e+00  1,4013e-45  7,4269e-44  2,7620e-42  9,9538e-41  3,4867e-39  1,1865e-37  3,9190e-36  1,2553e-34  3,8955e-33  1,1700e-31  3,3969e-30  9,5234e-29  2,57
48e-27  6,7044e-26  1,6788e-24  4,0360e-23  9,3004e-22  2,0503e-20  4,3153e-19  8,6522e-18  1,6484e-16  2,9763e-15  5,0771e-14  8,1549e-13  1,2286e-11  1,7288e-
10  2,2606e-09  2,7313e-08  3,0289e-07  3,0586e-06  2,7860e-05  2,2627e-04  1,6152e-03  9,9485e-03  5,1615e-02  2,1828e-01  7,1828e-01  1,7183e+00  2,7183e+00

EndErrors
2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,71
83e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+
00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7183e+00  2,7181e+00  2,7167e+00  2,7083e+00  2,6667e+00  2,5000e+00  2,0000e+00  1,0000e+00  0,0000e+00
```

Рис. 9.1.

9.1. Сразу можно заметить, что ряд монотонно убывает и сходится медленнее, чем у синуса или косинуса (~ в два раза), что логично, т.к. в формуле разложения у косинуса и синуса в знаменателе факториал от 2n, а не от n. Также, раз ряд не знакопеременный, суммы монотонно возрастают. Как и ожидалось, суммирование с конца даёт более точный результат, так как, начиная с малых значений, накапливается хвост, который не будет видеть сумма, полученная

```
StartErrors
1,0956e+03  1,0886e+03  1,0641e+03  1,0070e+03  9,0692e+02  7,6687e+02  6,0347e+02  4,4006e+02  2,9709e+02  1,8588e+02
1,0804e+02  5,8505e+01  2,9609e+01  1,4049e+01  6,2697e+00  2,6392e+00  1,0508e+00  3,9673e-01  1,4233e-01  4,8584e-02
1,5747e-02  4,7607e-03  1,3428e-03  2,4414e-04  1,2207e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04
2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04
2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04
2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04
2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04  2,4414e-04

EndErrors
1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03
1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03
1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03
1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03
1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03
1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0966e+03  1,0965e+03  1,0962e+03  1,0956e+03  1,0940e+03
1,0904e+03  1,0826e+03  1,0670e+03  1,0381e+03  9,8859e+02  9,1075e+02  7,9955e+02  6,5657e+02  4,9317e+02  3,2977e+02
1,8971e+02  8,9667e+01  3,2500e+01  8,0000e+00  1,0000e+00  0,0000e+00
```

Рис. 9.2.

суммированием с начала. Если дать на вход программы x = 7, то результат будет очень наглядным. Вывод представлен на Рис. 9.2. (часть вывода). Видно, что на 26 элементе *StartErrors* сумма перестала видеть последующие члены ряда, так как различия между суммой и членом слишком большие, в то же самое время суммирование с конца даёт очень точный результат, ошибка монотонно убывает, пока не становится примерно равной нулю, только есть проблема, что первые элементы *EndErrors* не меняются, хотя сумма копится, это связано с тем, что сумма ещё слишком мала, из-за чего её вычитание из эталонного значения не влияет на результат. Если строить график, то получится Рис. 9.3. На нём хорошо видно, как при

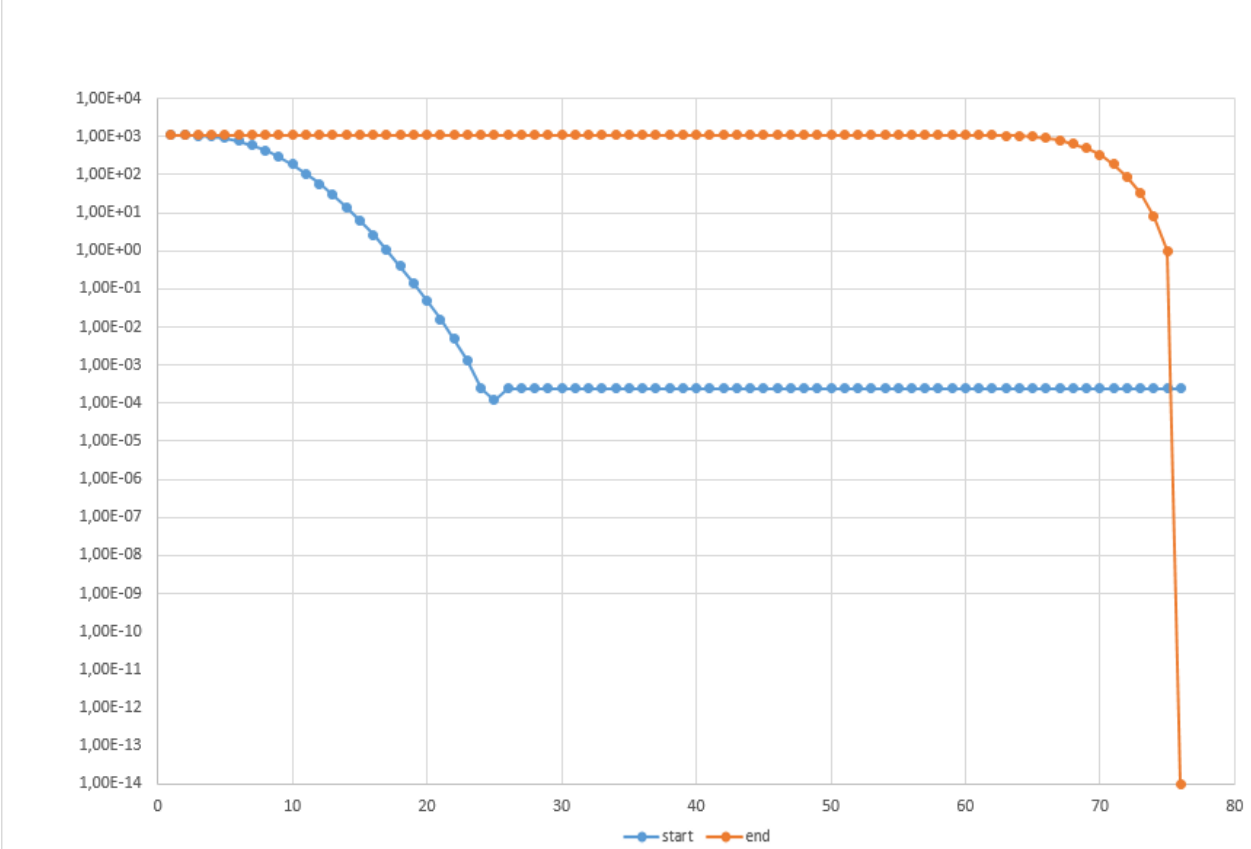

Рис. 9.3. График изменения ошибок для экспоненты при больших значениях x

суммировании с первых членов, ошибка монотонно убывает, но потом остаётся на одном значении. В то время как суммирование с конца даёт точный результат.

По экспоненте можно сделать вывод, что суммирование членов ряда с конца является лучшим алгоритмом, т.к. все члены ряда положительны, нет сложностей с суммами очень малых занчений, у которых разный знак, эксперименты подтвердили это утверждение.

Ln(1+x)

Логарифм интересен тем, что сходится очень медленно (медленнее чем все предыдущие функции), т.к. у него в знаменятеле нет факториала n, а просто номер текущего члена, также эта функция сходится только при |x|<1, иначе ряд будет просто расходится, что не имеет большого смысла. Ряд является знакопеременным, что тоже вызывает трудности с вычислениями. Для начала был выбран x = 0.1 для более быстрой сходимости ряда, результат вывода показан на Рис. 10.1. Видно, насколько медленно сходится ряд, для 0.1 уже

```
Ryad
0,0000e+00  1,0000e-01  -5,0000e-03  3,3333e-04  -2,5000e-05  2,0000e-06  -1,6667e-07  1,4286e-08  -1,2500e-09  1,1111e-10  -1,0000e-11  9,0909e-13  -8,3333e-14  7,69
23e-15  -7,1429e-16  6,6667e-17  -6,2500e-18  5,8824e-19  -5,5556e-20  5,2632e-21  -5,0000e-22  4,7619e-23  -4,5455e-24  4,3478e-25  -4,1667e-26  4,0000e-27  -3,8462e
-28  3,7037e-29  -3,5714e-30  3,4483e-31  -3,3333e-32  3,2258e-33  -3,1250e-34  3,0303e-35  -2,9412e-36  2,8571e-37  -2,7778e-38  2,7027e-39  -2,6316e-40  2,5641e-41
 -2,4999e-42  2,4383e-43  -2,3822e-44  2,8026e-45  -0,0000e+00

StartZnach
0,0953101814

StartSumm
0,0000e+00  1,0000e-01  9,5000e-02  9,5333e-02  9,5308e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02
  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-
02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310
e-02  9,5310e-02  9,5310e-02  9,5310e-02

StartErrors
9,5310e-02  4,6898e-03  3,1018e-04  2,3149e-05  1,8477e-06  1,4901e-07  1,4901e-08  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00
  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+
00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000e+00  0,0000
e+00  0,0000e+00  0,0000e+00  0,0000e+00

EndZnach
0,0953101814

EndSumm
-0,0000e+00  2,8026e-45  -2,1019e-44  2,2281e-43  -2,2771e-42  2,3364e-41  -2,3979e-40  2,4629e-39  -2,5315e-38  2,6040e-37  -2,6808e-36  2,7622e-35  -2,8488e-34  2,9
409e-33  -3,0392e-32  3,1444e-31  -3,2570e-30  3,3780e-29  -3,5084e-28  3,6492e-27  -3,8018e-26  3,9677e-25  -4,1487e-24  4,3470e-23  -4,5653e-22  4,8066e-21  -5,0749
e-20  5,3749e-19  -5,7125e-18  6,0954e-17  -6,5333e-16  7,0390e-15  -7,6294e-14  8,3280e-13  -9,1672e-12  1,0194e-10  -1,1481e-09  1,3138e-08  -1,5353e-07  1,8465e-06
  -2,3154e-05  3,1018e-04  -4,6898e-03  9,5310e-02  9,5310e-02

EndErrors
9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02
  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-
02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5310e-02  9,5308e-02  9,5333e-02  9,5000
e-02  1,0000e-01  0,0000e+00  0,0000e+00
```

Рис. 10.1.

понадобилось 45 членов ряда, чтобы последний из них занулился. Но в то же время стартовая сумма скорректировалась уже на 8 элементе, из-за чего на самом деле было достаточно ~ 10 элементов. Значения при двух алгоритмах суммирования получилось одинаковым, но в теории, сумма с конца должна быть более точной, т.к. ряд сразу монотонно убывает, в отличие от синуса и косинуса, и хвост из малых значений накапливается корректно, между малыми значениями нет таких больших «скачков», что не даёт ошибке так сильно накапливаться. Большим значением был выбран x = 0.6, результат работы программы показан на Рис. 10.2.

```
StartSumm
0,0000e+00  6,0000e-01  4,2000e-01  4,9200e-01  4,5960e-01  4,7515e-01  4,6738e-01  4,7138e-01  4,6928e-01  4,7040e-01  4,6979e-01  4,7012e-01  4,6994e-01  4,7004e-01  4,6998e-01
 4,7001e-01  4,7000e-01  4,7001e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01
  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-0
1  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-
01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e
-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000
e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,700
0e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,70
00e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7
000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,
7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4
,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01
4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01
 4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01  4,7000e-01

EndSumm
1,4013e-45  0,0000e+00  1,4013e-45  0,0000e+00  1,4013e-45  0,0000e+00  1,4013e-45  0,0000e+00  2,8026e-45  -1,4013e-45  5,6052e-45  -5,6052e-45  1,2612e-44  -1,6816e-44  3,2230e-
44  -5,0447e-44  8,6881e-44  -1,4433e-43  2,4242e-43  -4,0638e-43  6,8103e-43  -1,1407e-42  1,9128e-42  -3,2048e-42  5,3740e-42  -9,0061e-42  1,5099e-41  -2,5310e-41  4,2434e-41
-7,1137e-41  1,1927e-40  -1,9996e-40  3,3527e-40  -5,6216e-40  9,4263e-40  -1,5806e-39  2,6506e-39  -4,4450e-39  7,4545e-39  -1,2502e-38  2,0968e-38  -3,5168e-38  5,8988e-38  -9,8
945e-38  1,6598e-37  -2,7843e-37  4,6708e-37  -7,8361e-37  1,3147e-36  -2,2058e-36  3,7010e-36  -6,2102e-36  1,0421e-35  -1,7487e-35  2,9347e-35  -4,9252e-35  8,2663e-35  -1,3874e
-34  2,3288e-34  -3,9092e-34  6,5624e-34  -1,1017e-33  1,8496e-33  -3,1053e-33  5,2140e-33  -8,7550e-33  1,4702e-32  -2,4689e-32  4,1463e-32  -6,9639e-32  1,1697e-31  -1,9647e-31
 3,3004e-31  -5,5444e-31  9,3149e-31  -1,5650e-30  2,6297e-30  -4,4188e-30  7,4257e-30  -1,2480e-29  2,0975e-29  -3,5255e-29  5,9262e-29  -9,9625e-29  1,6749e-28  -2,8161e-28  4,7
351e-28  -7,9626e-28  1,3391e-27  -2,2522e-27  3,7882e-27  -6,3723e-27  1,0720e-26  -1,8036e-26  3,0347e-26  -5,1067e-26  8,5941e-26  -1,4464e-25  2,4347e-25  -4,0985e-25  6,9001e
-25  -1,1618e-24  1,9564e-24  -3,2947e-24  5,5492e-24  -9,3475e-24  1,5747e-23  -2,6532e-23  4,4708e-23  -7,5345e-23  1,2699e-22  -2,1407e-22  3,6090e-22  -6,0852e-22  1,0262e-21
 -1,7308e-21  2,9195e-21  -4,9254e-21  8,3109e-21  -1,4025e-20  2,3673e-20  -3,9963e-20  6,7475e-20  -1,1394e-19  1,9245e-19  -3,2511e-19  5,4931e-19  -9,2830e-19  1,5691e-18  -2,
6527e-18  4,4855e-18  -7,5864e-18  1,2834e-17  -2,1716e-17  3,6753e-17  -6,2217e-17  1,0535e-16  -1,7843e-16  3,0229e-16  -5,1227e-16  8,6835e-16  -1,4724e-15  2,4973e-15  -4,2369
e-15  7,1908e-15  -1,2208e-14  2,0733e-14  -3,5225e-14  5,9867e-14  -1,0179e-13  1,7314e-13  -2,9462e-13  5,0156e-13  -8,5425e-13  1,4557e-12  -2,4817e-12  4,2332e-12  -7,2247e-12
  1,2338e-11  -2,1081e-11  3,6045e-11  -6,1671e-11  1,0559e-10  -1,8092e-10  3,1024e-10  -5,3244e-10  9,1459e-10  -1,5725e-09  2,7064e-09  -4,6628e-09  8,0426e-09  -1,3889e-08  2,
4018e-08  -4,1591e-08  7,2131e-08  -1,2530e-07  2,1806e-07  -3,8022e-07  6,6440e-07  -1,1637e-06  2,0435e-06  -3,5987e-06  6,3581e-06  -1,1274e-05  2,0072e-05  -3,5903e-05  6,4564
e-05  -1,1683e-04  2,1298e-04  -3,9168e-04  7,2806e-04  -1,3715e-03  2,6276e-03  -5,1484e-03  1,0404e-02  -2,1996e-02  5,0004e-02  -1,3000e-01  4,7000e-01  4,7000e-01
```

Рис. 10.2

(часть вывода). Значения для двух типов суммирования получились одинаковыми. Но интереснее выглядит начало массива *EndSumm*, так как ряд у логарифма знакопеременный, то слагаемые чередуют знак, но в массиве мы получили: чередование числа и нуля, слагаемые ряда не занулились, а находятся на границе диапазона, из-за чего представляются как одно и то же число. То есть нельзя привести пример, где значение последнего элемента *EndSumm* будет точнее, так как точности float не хватит на малые значения членов ряда, чтобы накопилось достаточное значение, способное повлиять на сумму. График представлен на Рис. 10.3.

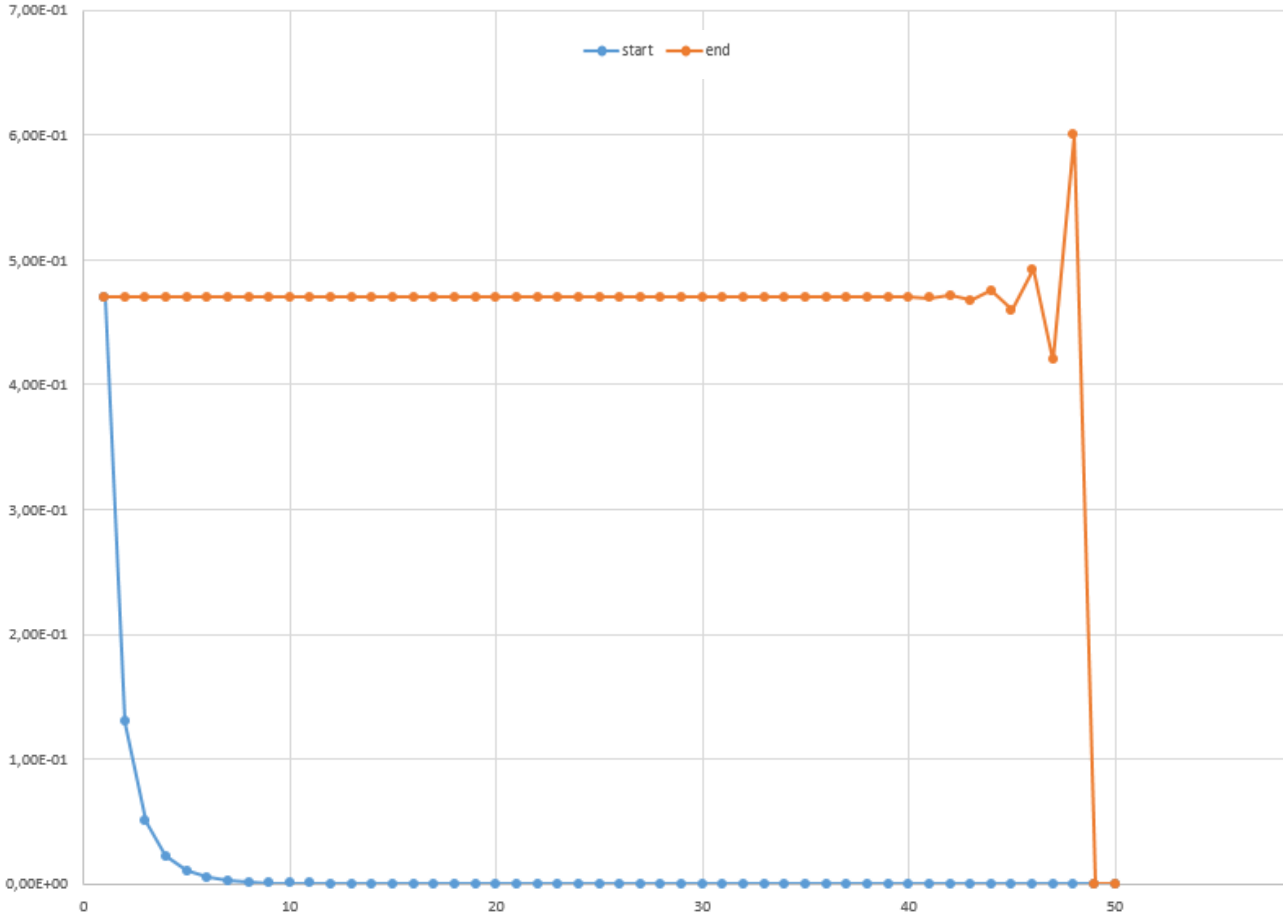

Рис. 10.3. График изменения ошибок для логарифма при больших значениях x

Для логарифма можно сделать вывод, что в типе float разница между суммированием с начала и конца настолько мала, что лежит в пределах машинной точности, для вычислений оба метода эквивалентны.

Заключение

Порядок суммирования членов ряда Тейлора существенно влияет на точность вычислений, особенно при работе с числами с плавающей точкой. Выбор оптимального метода зависит от поведения членов ряда (рост, убывание, знакопеременность) и от величины аргумента. Полученные результаты демонстрируют важность учёта особенностей хранения чисел с плавающей точкой.

Литература

1. Керниган Б., Ритчи Д., Фьюэр А. Язык программирования СИ //М.: Финансы и статистика. – 1992.

Приложение

```
void Fill(struct Xn* r, float(*function)(float prev, float x, int
n), float x)
{
	r->slagaemoe[0] = function(0.0f, x, 0);
	for (int i = 1; i < r->n; i++)
	{
		r->slagaemoe[i] = function(r->slagaemoe[i-1], x, i);
	}
}
void Errors(struct Xn* r, float(*etalon)(float x), float x)
{
	float et = etalon(x);
	for (int i = 0; i < r->n; i++)
	{
		r->starterr[i] = fabs(et - r->startsum[i]);
	}
	for (int i = 0; i < r->n; i++)
	{
		r->enderr[i] = fabs(et - r->endsum[i]);
	}
}
void Sum(struct Xn* r)
{
	r->endsum[0] = r->slagaemoe[r->n - 1];
	r->startsum[0] = r->slagaemoe[0];
	for (int i = 1; i < r->n; i++)
	{
		r->startsum[i] = r->slagaemoe[i] + r->startsum[i - 1];
	}
	for (int i = 1; i < r->n; i++)
	{
		r->endsum[i] = r->slagaemoe[r->n -1 -i] + r->endsum[i
- 1];
	}
}
void choices(int choice, struct Xn* r, float x)
{
	float(*function)(float prev, float x, int n) = NULL;
	float(*etalon)(float x) = NULL;
	switch (choice)
	{
	case 1: function = rSin; etalon = sinf; break;
	case 2: function = rCos; etalon = cosf; break;
	case 3: function = rExp; etalon = expf; break;
	case 4: function = rLn; etalon = log1pf; break;
	default: printf("Error"); return;
	}
	Fill(r, function, x);
	Sum(r);
	Errors(r, etalon, x);
}
```